

Assignment 2

1.

a. ① DLP: Data-Level Parallelism.

many data items being ~~oper~~ operated on at the same time.

② TLP: Task-Level Parallelism

tasks of work created to operate independently and largely in parallel.

b. ① ILP: Instruction-Level Parallelism

exploits data-level parallelism at modest levels -
- pipelining, ~~also~~ divide a task to steps,
simultaneously run different steps of different tasks,
and do some work in advance.

② Vector Architectures & GPUs & Multimedia Instruction Sets

exploits data-level parallelism; apply a single instruction to a collection of data in parallel.

③ TLP: Thread-Level parallelism

exploits either DLP or TLP in a tightly coupled hardware model that allows for interaction among parallel threads.



④ RLP = Request - Level Parallelism.

exploits parallelism among largely decoupled tasks specified by the programmer ^{or} ~~on~~ the OS.

c.

DVFS = dynamic voltage-frequency scaling

scale down clock ~~and~~ frequency and voltage during periods of low activity

Power_{dynamic} $\propto \frac{1}{2} \times \text{Capacitive load} \times \text{Voltage}^2 \times \text{frequency switched}$

d. overclocking - Turbo mode.

the chip ~~has~~ runs at a higher clock rate for a short time until temperature rises.



2.

$$\text{Original CPI} = 0.25 \times 4.0 + 0.75 \times 1.33 = 1.9975.$$

design alternative 1:

$$\text{new CPI}_1 = 0.25 \times 4 - 0.02 \times 20 + 0.02 \times 2 + 0.75 \times 1.33 = 1.6375$$

$$\therefore \text{speedup}_1 = \frac{1.9975}{1.6375} \approx 1.22$$

design alternative 2:

$$\text{new CPI}_2 = \cancel{0.25 \times 4} + \cancel{0.75 \times 0.5}$$

$$0.25 \times 2.5 + 0.75 \times 1.33 = 1.6225$$

$$\therefore \text{speedup}_2 = \frac{1.9975}{1.6225} \approx 1.23$$

$$\therefore \text{Speedup}_2 > \text{Speedup}_1$$

Although the FPSRR optimization offers 10x improvement, FPSRR constitutes only 2% of instructions. In contrast, alternative 2 improves all FP instructions (25% of all), consistent with Amdahl's law: optimizing a larger fraction of execution yields greater overall speedup.



3. the number of blocks = $128\text{ B} \div 4\text{ B/block}$
= 32 blocks.

a. the number of instructions = $64\text{ B} \div 4\text{ B} = 16\text{ Instructions}$
 $\therefore$ all 16 instructions fit entirely in the cache.

On the first iteration: 16 compulsory miss

After that, every subsequent iteration hits for all 16 instructions

c. Asymptotic miss rate = 0%.

b.

① 192-byte loop

the number of instruction = $192\text{ B} \div 4\text{ B} = 48\text{ instructions}$.

$\therefore$ During the execution, when we access the instruction

33, the cache is full and the LRU policy evicts instruction 1. Instruction 34 evicts instruction 2, and

so on. By the time, we finish the first loop and the cache holds instructions 17-48

when the loop wraps around to instruction 1,

instruction 1 evicts instruction 17

Therefore, every single access is a miss.



the asymptotic miss rate = 100%.

② 320-byte loop.

the number of instructions = $320B \div 4B = 80$ instructions.

The same pathological behavior occurs.

With 80 instructions and only 32 cache blocks, LRU evicts exactly the instructions that will be needed next as the loop wraps around.

$\therefore$ asymptotic rate = 100%.

c.

① 64-byte loop

The cache already holds all 16 instructions

miss rate = 0%.

$\therefore$ MRU provides no benefit.

② 192-byte loop. 148 instructions.

instruction ~~32~~ 33 evicts instruction 32 under MRU

instruction 34 evicts instruction 33 and so on.

when we reach instruction 48. the cache holds

instruction 1-31 and instruction 48.



When the loop wraps around instruction 1-31 all hit and instruction 32-48 miss.

$$\therefore \text{miss rate} = (48 - 31) \div 48 \times 35.4\% < 100\%$$

③. 480-byte loop (80 instructions)

By the same logic instruction 1-31 are retained and hit while instruction 32-80 each miss.

$$\therefore \text{miss rate} = (80 - 31) \div 80 = 61.25\% < 100\%$$

therefore, the 192-byte loop and 320-byte loop cases benefit from MRU replacement.

The 64-byte loop does not benefit from MRU because LRU already achieves 0% miss rate.



4.

a. $\therefore \text{AMAT} = \text{hit time} + \text{miss rate} \times \text{miss penalty}$

$$= 1 + 0.05 \times 105 = 6.25 \text{ cycles.}$$

b. $\text{hit rate} = 64\text{KB} \div 256\text{MB} \approx 0.0244\%$

$$\therefore \text{miss rate} = 1 - \text{hit rate} \approx 99.976\%$$

$$\text{AMAT} = 1 + 0.99976 \times 105 \approx 105.97 \text{ cycles}$$

without locality, the cache is not only useless but actually harmful due to the miss penalty overhead.

c.

$$\text{cache hit } G = 99 \text{ cycles}$$

$$\text{cache miss } L = 5 \text{ cycles.}$$

$$\therefore \text{AMAT}_{\text{cache}} = (1-m) \times 1 + m \times 105 = 1 + 104m$$

$$\text{AMAT}_0 = 100$$

$$\text{when } 1 + 104m > 100 \quad m > \frac{99}{104} \approx 95.2\%$$

cache becomes ~~dis~~disadvantageous

In other word.

$$(1-m) \times G > m \times L \quad m > \frac{G}{G+L} \approx 95.2\%$$

therefore, the highest miss rate is 95.2%.



5. Data words = 64 bit = 8B

$\therefore$ Each block = $16 \times 8B = 128B$

$\therefore$ words are addressed to half-word

$\therefore$ Addressable units per block = $128B \div 4B = 32$

$\therefore$ offset = $\log_2 32 = 5$ bits

total blocks = $2MB \div 128B = 2^{14}$ blocks

number of sets = $2^{14} \div 16 \text{ ways} = 2^{10}$ sets

$\therefore$ index = $\log_2 2^{10} = 10$ bits

tag = $14 - 10 = 4$ bits

